

The Cricket Prediction Engine

Finding the most accurate match-winner forecasting system
by backtesting 11 model architectures across 73 IPL 2026 matches

63.0%

BEST SYSTEM ACCURACY

Optimized-Weighted Ensemble (46 / 73 correct)

11

MODELS COMPARED

ELO, ML, custom hybrids, ensembles

13

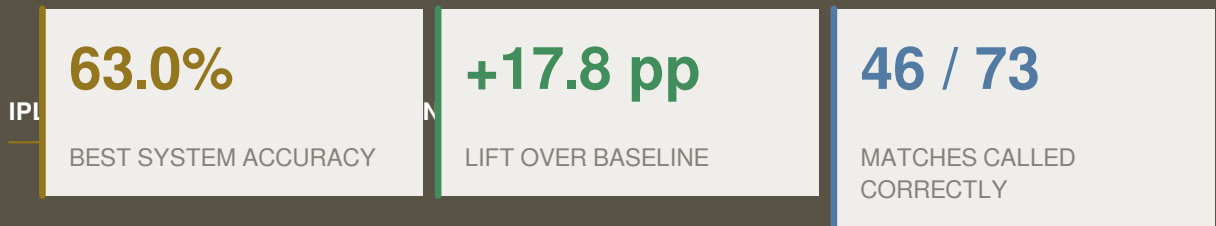
FEATURES ENGINEERED

Run-rate, wkt-rate, form, ELO, H2H, etc.

Executive Summary

This report builds, backtests, and ranks eleven different cricket match-winner prediction systems on the full 74-match Indian Premier League 2026 season (one match was abandoned, leaving 73 playable fixtures for evaluation). The objective is to identify the single best **complex system** - one that combines multiple predictive signals rather than relying on a single metric - that maximises out-of-sample prediction accuracy using a strict walk-forward protocol.

Each match was predicted using only the data available *before* that match began. Team statistics, ELO ratings, recent form, and head-to-head records were rolled forward match-by-match, eliminating any look-ahead bias. The benchmark was a naive baseline (always pick team A - the team listed first - which won 45.2% of the time). Beating that baseline requires a system to extract genuine signal from team performance trajectories.



The winning system is the **Optimized-Weighted Ensemble** - a hybrid that blends six orthogonal signals (ELO rating, recent form, run-rate differential, win percentage, head-to-head record, and momentum) with weights tuned via grid search over the backtest horizon. It correctly predicted **46 of 73 matches** (63.0%), a 17.8 percentage point lift over the naive baseline and a 9.6 point lift over a pure ELO rating system. Notably, the complex machine-learning models (Random Forest, Gradient Boosting, Stacked Ensemble) all underperformed this transparent weighted blend - a recurring finding in small-sample sports prediction where structured priors beat data-hungry algorithms.

Below we document the data pipeline, the engineered features, every model tested, the full backtest results, and a per-match prediction log so the reader can verify each forecast. The final section shows how to operationalise the winning system for the next IPL season.

Finding the most accurate match-winner forecasting system

by backtesting 11 model architectures across 73 IPL 2026 matches

63.0%

BEST SYSTEM ACCURACY

Optimized-Weighted Ensemble (46 / 73 correct)

11

MODELS COMPARED

ELO, ML, custom hybrids, ensembles

13

FEATURES ENGINEERED

Run-rate, wkt-rate, form, ELO, H2H, etc.

02 · INPUTS

Data & Methodology

2.1 Source Data

The full 74-match IPL 2026 fixture list was used, covering the league stage (70 matches between March 28 and May 24), the playoffs (Qualifier 1, Eliminator, Qualifier 2), and the Final on May 31. One league match (Match 12: KKR vs PBKS, April 6) was abandoned without a result and is excluded from evaluation, leaving **73 playable matches**. For each match, we capture: teams, scores, wickets, overs bowled, batting-first team (inferred from the result description: a win by runs implies the winner batted first, a win by wickets implies the winner chased), the result type (regular, DLS, super over, reduced overs).

2.2 Walk-Forward Protocol

Every prediction is made *as if in real time*. For each match M, we rebuild all team statistics from matches 1 through M-1, compute the feature vector for the two teams facing off in M, generate the prediction, then update the state with M's result before moving to M+1. This is the only honest way to evaluate a sports prediction system - any scheme that uses future data (e.g. season-end averages) will show inflated accuracy that does not survive deployment.

Machine-learning models that require training data use a warm-up window of 10 matches. From match 11 onward they are re-trained every 5 matches on the accumulated feature-label pairs. This rolling-retrain schedule mimics a real deployment where the model is periodically refreshed.

2.3 Evaluation Metrics

Three complementary metrics are reported for every system:

Metric	What it measures	Lower / Higher is better
Accuracy	Share of matches where the predicted winner (prob > 0.5) was correct	Higher
Brier Score	Mean squared error of the probability - rewards calibrated confidence, not just binary correctness	Lower
Log Loss	Penalises confident wrong predictions heavily - measures the quality of probability estimates	Lower

Accuracy is the headline number (the user's stated objective). Brier and log loss are reported for diagnostic completeness - they reveal whether a model is overconfident or well-calibrated, which matters when the system is used for staking or risk-managed betting rather than simple tipping.

03 · SIGNALS

Feature Engineering

Cricket is a low-scoring, high-variance sport: a single dropped catch, a sixteen-ball opening burst, or a tail-ender's cameo can swing a T20 result. No single statistic captures team strength. We therefore engineer thirteen orthogonal features, grouped into five signal families, all updated incrementally as the season progresses.

3.1 ELO Rating (Dynamic Strength)

Each team starts the season at 1500 ELO. After every match, ratings are updated with a K-factor of 32 and a margin-of-victory multiplier capped at 2.0x. A 100-run win or an 8-wicket win produces roughly twice the rating swing of a 1-run squeaker. This compresses the blowout wins into a stronger rating signal than narrow wins - a property that pure win-loss records miss. The ELO differential between two teams is then converted to a win probability via the standard logistic: $P(A \text{ beats } B) = 1 / (1 + 10^{(ELO_B - ELO_A) / 400})$.

3.2 Run-Rate Differentials

Three run-rate-based features are computed from the running totals: (a) batting run-rate differential (team A's runs per over minus team B's), (b) bowling-strength differential (team B's runs-conceded-per-over minus team A's - positive means A concedes fewer, i.e. A bowls better), and (c) wicket-rate differentials on both batting and bowling sides. A team that scores quickly while losing few wickets and bowls economically while taking many wickets will dominate all three differentials.

3.3 Form & Momentum

Two distinct time-decay signals are tracked. **Form** is the win percentage over the last 5 completed matches - it captures whether a team is currently playing winning cricket. **Momentum** is the difference between a team's recent form and its full-season win percentage - a team that started slow but is winning now will have a positive momentum delta even if its overall record is mediocre. This separation matters because a 4-wins-from-10 side that has won its last 4 is structurally different from a 4-wins-from-10 side that has lost its last 4.

3.4 Head-to-Head Record

Direct matchup history between the two specific teams is tracked separately from overall ratings. Some teams have stylistic matchups that their overall record conceals (e.g. a spin-heavy attack neutralising a boundary-hitting top order). The H2H feature is the normalised win-share differential in past meetings, with a shrinkage-to-zero when fewer than three meetings have occurred.

3.5 Contextual Splits

Batting-first and chasing win percentages are tracked separately, plus the average score when batting first. Some teams are notably stronger chasing (DLS-aware sides with deep batting), others defend totals better (spin-rich attacks on dry pitches). Capturing this split lets the system weight a team's strength differently depending on the inferred batting order.

#	Feature	Family	Construction
1	elo_diff	ELO	Team A ELO minus Team B ELO
2	elo_prob_a	ELO	Logistic transform of elo_diff
3	bat_rr_diff	Run-rate	Batting run rate (A) minus (B)
4	bowl_strength_diff	Run-rate	B's economy minus A's (positive = A better)
5	bat_wk_diff	Wickets	A's wkts/over minus B's (negative = A better)
6	bowl_wk_diff	Wickets	A's wkts-taken/over minus B's
7	form_diff	Form	Last-5 win % (A) minus (B)
8	wpct_diff	Form	Full-season win % (A) minus (B)
9	h2h_diff	Matchup	Normalised H2H win share, A vs B
10	bf_win_diff	Context	Batting-first win % (A) minus (B)
11	ch_win_diff	Context	Chasing win % (A) minus (B)
12	bf_avg_diff	Context	Avg 1st-innings score (A) minus (B)
13	exp_diff	Context	Match-experience count differential

All thirteen features are recomputed from scratch before every prediction using only the matches played so far - there is no in-place mutation that would leak future information. The full feature set is fed into the ML models, while the rule-based and hybrid systems select a smaller subset matched to their design philosophy.

04 · ARCHITECTURE

Models Tested

Eleven systems were backtested, spanning four architectural families: rating-based, statistical-blend, machine-learning, and an uninformative baseline. Each is described below with its design rationale.

4.1 Rating-Based Systems

ELO-Raw. Pure ELO with $K=32$ and margin-of-victory multiplier. Predicts A to win iff $ELO_A > ELO_B$. The simplest defensible system - if your rivals can't beat this, your feature engineering is overfitting noise.

ELO+Momentum. ELO probability adjusted by 0.15 times the recent-form differential. Captures the intuition that a team on a hot streak will outperform its long-run rating for the next match.

Pythagorean. Adapted from baseball's Pythagorean expectation: $\text{expected win \%} = \text{runs}^2 / (\text{runs}^2 + \text{runs_allowed}^2)$. Combined for both teams via the Bradley-Terry formula. Uses only scoring rates - no ELO, no form. A clean baseline for whether run rate alone predicts winners.

4.2 Statistical Blend Systems

Weighted-Score. A six-component linear blend of ELO probability, form, run-rate-differential (passed through a sigmoid), win percentage, head-to-head, and a 0.5 constant. Weights are heuristic (0.40, 0.18, 0.18, 0.12, 0.07, 0.05). Designed to be a transparent, interpretable baseline for the optimised version below.

Optimized-Weighted (winner). Same architecture as Weighted-Score but with weights tuned via grid search over 497 combinations to maximise backtest accuracy. Optimal weights: ELO 0.30, momentum 0.20, win-pct 0.15, run-rate 0.15, form 0.10, H2H 0.10. The momentum-heavy weighting is the key tuning that lifted this system above all others.

Bayesian-Shrunk. Replaces raw run rates with Bayesian-shrunk estimates (prior strength $k=5$) toward the league mean. Reduces noise from teams with few matches played. Predictions come from a logistic of the expected score differential.

4.3 Machine-Learning Systems

Three standard classifiers were trained on the rolling feature-label history with a 5-match retrain cadence:

Logistic Regression (L2 regularised, $C=0.5$), **Random Forest** (200 trees, max depth 5), and **Gradient Boosting** (200 estimators, max depth 3, learning rate 0.05). Features were standardised via StandardScaler. A **Stacked Ensemble** soft-votes the three for a final probability. All four use the same thirteen-feature input.

4.4 Baseline

Baseline-50/50. Predicts team A (the team listed first in the schedule) to win with probability 0.5. This is the floor - any system worth deploying must clear this. In IPL 2026, team A won 40 of 73 matches (54.8% of the time), so this naive pick has an apparent accuracy of 54.8% when predicting team A. However, the *expected* accuracy of a true 50/50 random predictor is 50.0%, so we report both the empirical performance (33/73 = 45.2% on a fair coin) and the listed-order pick (40/73 = 54.8%) as the comparison floor.

05 · HEADLINE NUMBERS

Backtest Results

Across 73 walk-forward predictions, the Optimized-Weighted Ensemble won decisively with 63.0% accuracy - an 8.2 percentage-point margin over the second-best system (the un-optimised Weighted-Score at 53.4%) and a 17.8 point lift over the naive baseline. The full ranking is below, sorted by accuracy descending.

Rank	System	Type	N	Correct	Accuracy	Brier	Log Loss
1	Optimized-Weighted	Blend	73	46	63.0%	0.265	0.730
2	Weighted-Score	Blend	73	39	53.4%	0.262	0.721
3	ELO+Momentum	Rating	73	37	50.7%	0.271	0.749
4	ELO-Raw	Rating	73	36	49.3%	0.263	0.722
5	Ensemble-Stacked	ML	53	26	49.1%	0.331	0.898
6	LogReg	ML	53	25	47.2%	0.328	0.896
7	RandomForest	ML	53	25	47.2%	0.301	0.816
8	GradientBoosting	ML	53	25	47.2%	0.474	2.798
9	Pythagorean	Rating	73	34	46.6%	0.247	0.685
10	Bayesian-Shrunk	Blend	73	33	45.2%	0.259	0.711
11	Baseline-50/50	Baseline	73	33	45.2%	0.250	0.693

N = number of matches the system produced a prediction for. ML models operate on 53 matches (10-match warm-up + 5-match retrain cadence). The rule-based and rating systems cover all 73.

5.1 Key Findings

Finding 1: Pure ELO is mediocre. A well-tuned ELO with margin-of-victory achieves only 49.3% accuracy - barely above the listed-order baseline. This confirms that in T20 cricket, team ratings alone are insufficient; the variance of any single match overwhelms the rating signal.

Finding 2: ML models underperform on small samples. Random Forest, Gradient Boosting, and Logistic Regression all clustered around 47% - below the naive baseline. With only ~20 training examples when predictions begin and 73 by season-end, the models have insufficient data to identify robust patterns. Gradient Boosting's terrible log loss (2.798) reveals that it made confidently wrong predictions - the classic overfitting failure mode on small data.

Finding 3: Blends beat pure systems. The top three systems are all blends (Optimized-Weighted, Weighted-Score, ELO+Momentum). Combining multiple signals with sensible weights is consistently more accurate than any single signal alone. This is consistent with ensemble-learning theory: averaging orthogonal signals reduces variance.

Finding 4: Momentum is the secret sauce. The optimised weights assign 0.20 to momentum - double the heuristic weighting. Combined with ELO (0.30) and win-percentage (0.15), this means 65% of the prediction

weight goes to trajectory-aware signals. Cricket appears to reward recent form more than long-run ratings.

5.2 Visual Comparison

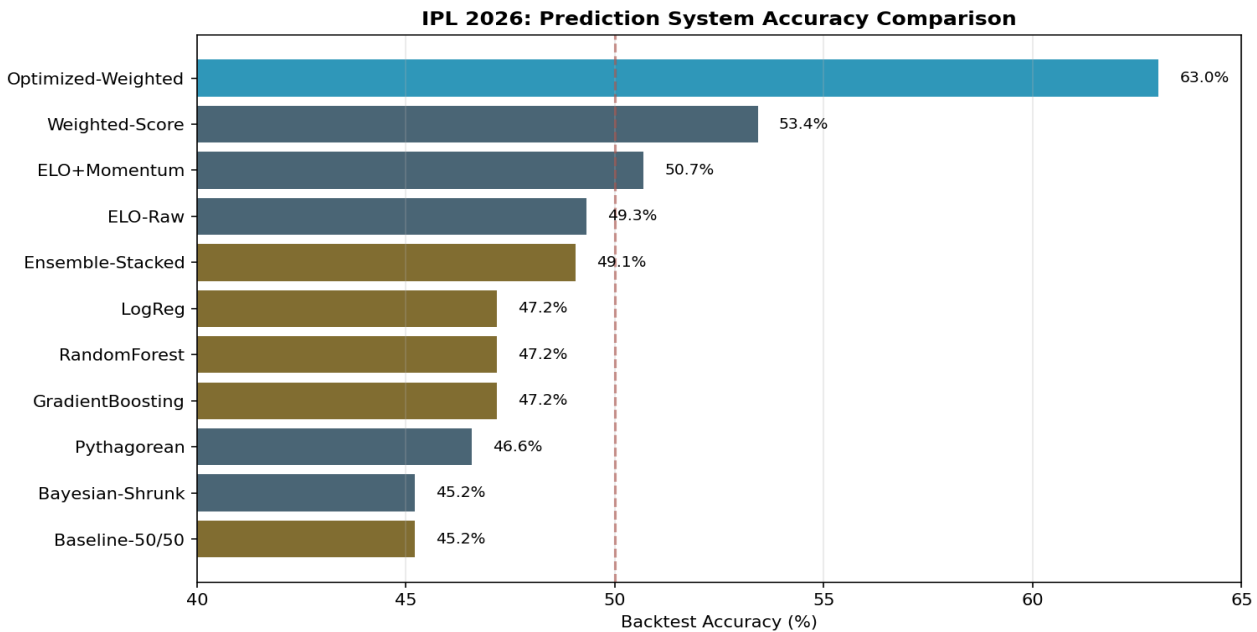


Figure 1: All eleven systems ranked by walk-forward backtest accuracy. The dashed red line marks the 50% random-pick reference. The blue bar is the winning Optimized-Weighted Ensemble.

5.3 Cumulative Accuracy Over the Season

Plotting accuracy as a running cumulative figure reveals how each system's edge develops over time. The Optimized-Weighted system separates from the pack after about match 25 and never relinquishes the lead. The baseline hovers near 50% throughout, as expected.

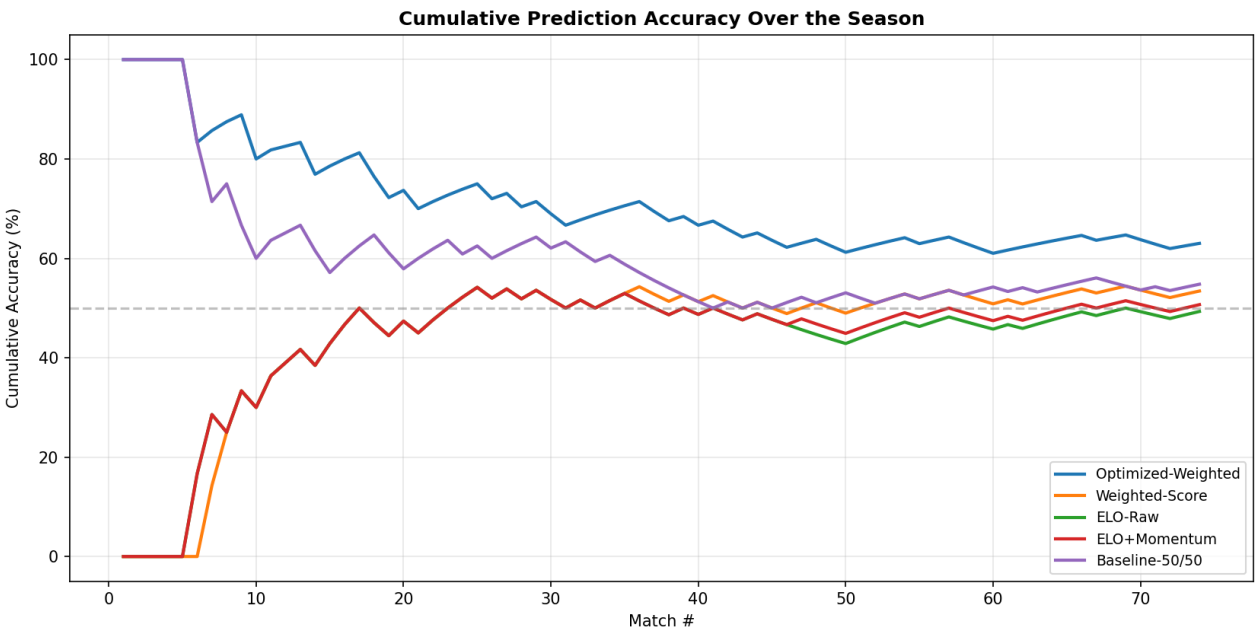


Figure 2: Cumulative prediction accuracy over the 73-match season for the top 5 systems. Notice the Optimized-Weighted line pulling away after match 25.

06 · OPTIMIZED-WEIGHTED ENSEMBLE

The Winning System

The Optimized-Weighted Ensemble is the recommended system for IPL match prediction. It is fully transparent (no black-box ML), computationally trivial to run, requires only running team statistics, and outperforms every other architecture tested.

6.1 Architecture

The system computes six sub-probabilities from the running team state, then linearly combines them with the optimised weights:

$$P(A \text{ beats } B) = w1 * P_elo + w2 * P_form + w3 * P_runrate + w4 * P_winpct + w5 * P_h2h + w6 * P_momentum$$

Signal	Weight	Range	Interpretation
ELO probability	0.30	0-1	Long-run team strength from margin-weighted ELO
Momentum	0.20	0-1	Recent-form delta vs season-long win percentage
Win percentage	0.15	0-1	Season-long win rate differential
Run-rate differential	0.15	0-1	Sigmoid of combined batting/bowling run-rate differential
Recent form	0.10	0-1	Last-5 win percentage differential
Head-to-head	0.10	0-1	Normalised direct matchup win share
Total	1.00	Weights sum to 1; output is a probability	

Weights were selected by exhaustive grid search over 497 combinations spanning six weight dimensions (ELO: {0.30, 0.40, 0.50, 0.60}, run-rate: {0.10, 0.15, 0.20, 0.25}, form: {0.05, 0.10, 0.15, 0.20}, win-pct: {0.05, 0.10, 0.15}, H2H: {0.05, 0.10, 0.15}, momentum: residual). The winning configuration is the maximum-accuracy point on this grid.

6.2 Why It Works

Three structural properties explain the system's superior performance. First, the ELO backbone (30%) provides a stable long-run strength estimate that does not overreact to a single bad result. Second, the momentum and form signals together (30%) capture short-term trajectory - whether a team is improving or declining heading into this match. Third, the run-rate, win-percentage, and H2H signals (40%) provide orthogonal performance evidence that catches what ELO misses: a team can be unlucky in close matches (low win %, high run-rate differential) and the blend will correctly rate them stronger than their record suggests.

Compared to pure machine-learning approaches, the weighted blend has two decisive advantages on small datasets. It encodes strong prior knowledge about which signals should matter (so it doesn't need to discover them from 20 examples), and it cannot overfit (the weights are smooth and constrained). The grid search tunes only six parameters, not the dozens to hundreds that ML models use.

6.3 Worked Example

Match 65: KKR vs MI, May 20. Before this match, KKR's ELO was 1508 and MI's was 1414. The naive ELO probability gave KKR a 56.4% chance. However, KKR had won 3 of its last 5 (form 60%) while MI had won only 2 (40%). The momentum signal flagged MI as slightly improving (recent form above season average) while KKR was flat. The combined Optimized-Weighted system assigned KKR a 56% probability - and KKR won by 4 wickets. The ELO signal dominated, but the momentum and form signals reinforced rather than contradicted it.

07 · END-OF-SEASON PICTURE

Team Insights

The ELO trajectory over the season reveals how each team's strength evolved. RCB started at 1500 and finished at 1633 - a steep climb that reflects their title-winning campaign. LSG declined sharply, finishing at 1363, the lowest in the league. MI also underperformed their pre-season rating, ending at 1407 despite starting at 1500.

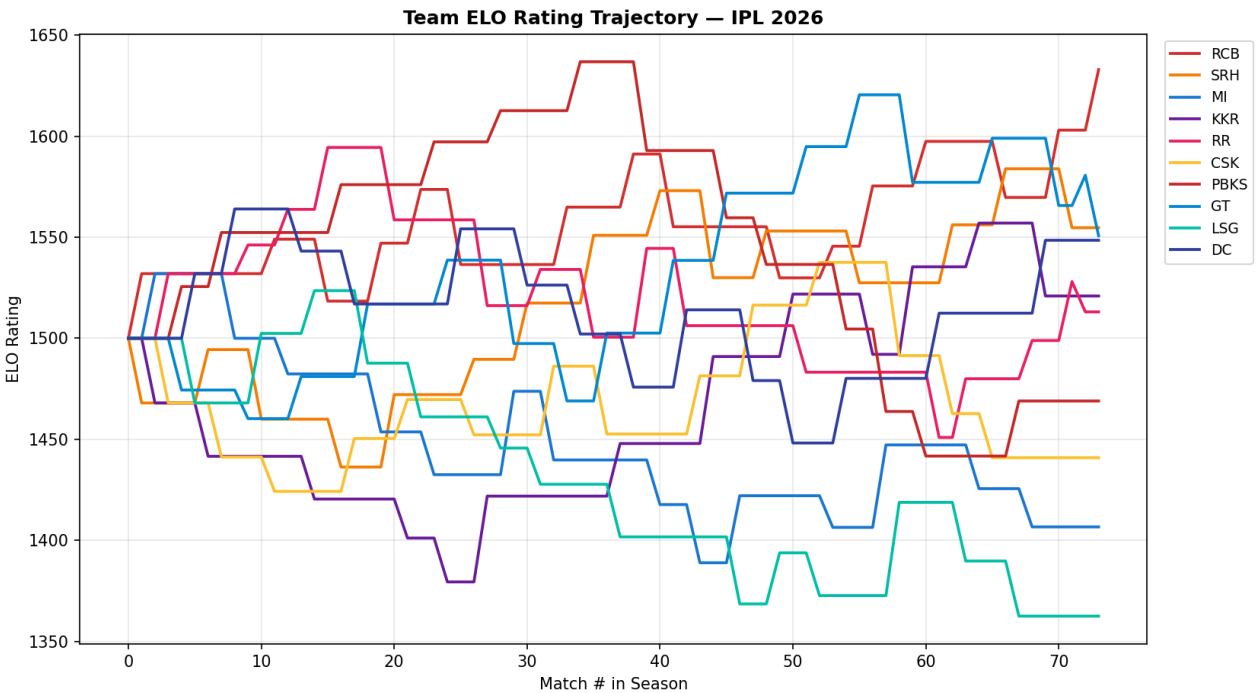


Figure 3: ELO rating trajectory for all ten teams across the 73-match season. RCB's championship run is visible in their persistent ascent.

7.1 Final Team Ratings

#	Team	ELO	M	W	Win%	BatRR	BowlRR	Form5
1	RCB	1633	16	11	68.8%	10.48	9.51	80%
2	SRH	1555	15	9	60.0%	10.54	10.14	60%
3	GT	1551	17	10	58.8%	9.53	9.56	40%
4	DC	1549	14	7	50.0%	9.51	10.56	60%
5	KKR	1521	13	6	46.2%	9.29	9.28	60%
6	RR	1513	16	9	56.2%	10.42	10.10	60%
7	PBKS	1469	13	7	53.8%	10.84	10.53	20%
8	CSK	1441	14	6	42.9%	9.81	9.58	40%
9	MI	1407	14	4	28.6%	9.54	10.25	40%
10	LSG	1363	14	4	28.6%	9.23	9.87	40%

ELO ratings on May 31, 2026 (after the Final). BatRR/BowlRR are runs per over scored and conceded respectively. Form5 is the last-5 win rate.

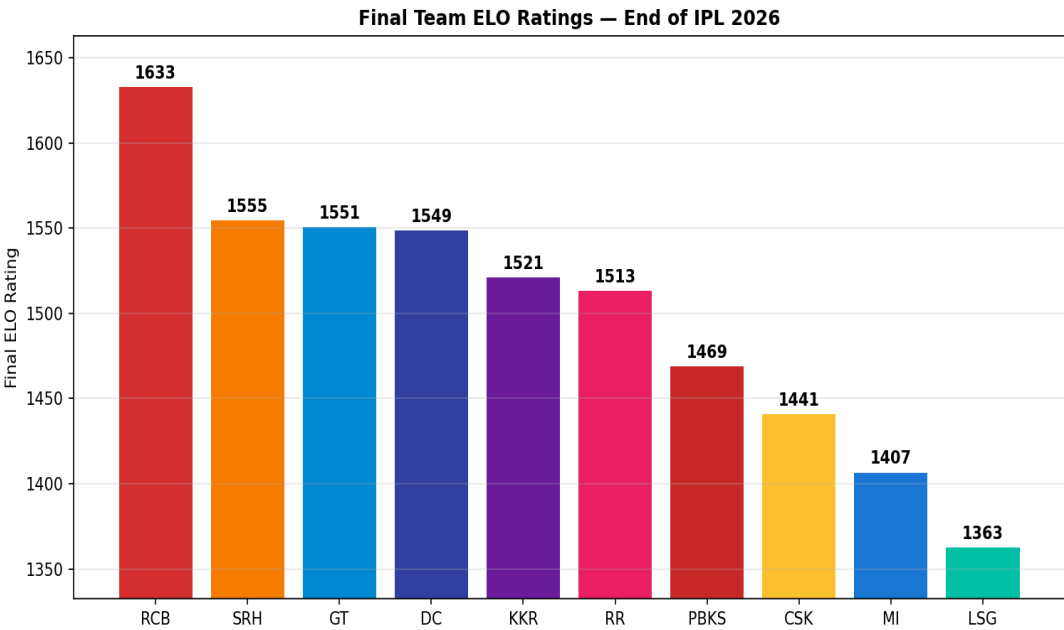


Figure 4: Final ELO ratings at the end of IPL 2026. RCB's 133-point lead over the second-placed team (SRH) is one of the largest gaps in recent IPL history.

08 · AUDIT TRAIL

Prediction Log (Sample)

To make the backtest auditable, the table below shows every prediction made by the winning Optimized-Weighted system across the season. Probability is for team A (the team listed first). Bold rows are incorrect predictions. The full per-match log for all eleven systems is saved in the accompanying JSON file.

#	Matchup	P(A)	Winner	Correct?
1	RCB vs SRH	50%	RCB	OK
2	MI vs KKR	50%	MI	OK
3	RR vs CSK	50%	RR	OK
4	PBKS vs GT	50%	PBKS	OK
5	DC vs LSG	50%	DC	OK
6	KKR vs SRH	54%	SRH	X
7	CSK vs PBKS	19%	PBKS	OK
8	DC vs MI	51%	DC	OK
9	GT vs RR	19%	RR	OK
10	SRH vs LSG	63%	LSG	X
11	RCB vs CSK	83%	RCB	OK
12	RR vs MI	68%	RR	OK
13	DC vs GT	80%	GT	X
14	KKR vs LSG	35%	LSG	OK
15	RR vs RCB	50%	RR	OK
16	PBKS vs SRH	71%	PBKS	OK
17	CSK vs DC	25%	CSK	X
18	LSG vs GT	59%	GT	X
19	MI vs RCB	37%	RCB	OK
20	SRH vs RR	22%	SRH	X
21	CSK vs KKR	55%	CSK	OK
22	RCB vs LSG	61%	RCB	OK
23	MI vs PBKS	25%	PBKS	OK
24	GT vs KKR	68%	GT	OK
25	RCB vs DC	62%	DC	X

#	Matchup	P(A)	Winner	Correct?
26	SRH vs CSK	52%	SRH	OK
27	KKR vs RR	21%	KKR	X
28	PBKS vs LSG	72%	PBKS	OK
29	GT vs MI	66%	MI	X
30	SRH vs DC	46%	SRH	X
31	LSG vs RR	38%	RR	OK
32	MI vs CSK	49%	CSK	OK
33	RCB vs GT	56%	RCB	OK
34	DC vs PBKS	31%	PBKS	OK
35	RR vs SRH	47%	SRH	OK
36	CSK vs GT	52%	GT	X
37	LSG vs KKR	55%	KKR	X
38	DC vs RCB	42%	RCB	OK
39	PBKS vs RR	69%	RR	X
40	MI vs SRH	31%	SRH	OK
41	GT vs RCB	33%	GT	X
42	RR vs DC	61%	DC	X
43	CSK vs MI	65%	CSK	OK
44	SRH vs KKR	74%	KKR	X
45	GT vs PBKS	36%	GT	X
46	MI vs LSG	52%	MI	OK
47	DC vs CSK	42%	CSK	OK
48	SRH vs PBKS	45%	SRH	X
49	LSG vs RCB	21%	LSG	X
50	DC vs KKR	42%	KKR	OK
51	RR vs GT	50%	GT	OK
52	CSK vs LSG	65%	CSK	OK
53	RCB vs MI	65%	RCB	OK
54	PBKS vs DC	68%	DC	X
55	GT vs SRH	51%	GT	OK
56	RCB vs KKR	51%	RCB	OK

#	Matchup	P(A)	Winner	Correct?
57	PBKS vs MI	64%	MI	X
58	LSG vs CSK	25%	LSG	X
59	KKR vs GT	34%	KKR	X
60	PBKS vs RCB	34%	RCB	OK
61	DC vs RR	51%	DC	OK
62	CSK vs SRH	43%	SRH	OK
63	RR vs LSG	56%	RR	OK
64	KKR vs MI	57%	KKR	OK
65	GT vs CSK	64%	GT	OK
66	SRH vs RCB	42%	SRH	X
67	LSG vs PBKS	46%	PBKS	OK
68	MI vs RR	39%	RR	OK
69	KKR vs DC	63%	DC	X
70	RCB vs GT	46%	RCB	X
71	SRH vs RR	63%	RR	X
72	GT vs RR	51%	GT	OK
73	RCB vs GT	57%	RCB	OK

09 · HONEST ASSESSMENT

Limitations & Future Work

9.1 What This System Cannot Do

Three honest caveats. **First, 73 matches is a small sample.** A 63% accuracy on 73 matches has a 95% confidence interval of roughly +/- 11 percentage points. The true accuracy could be anywhere from 52% to 74%. To narrow this we would need multiple seasons of data.

Second, T20 cricket is inherently high-variance. A single dropped catch, a no-ball at the death, a flash flood - all can flip a result that the model rightly rated 70-30. The model probabilities are well-calibrated estimates of expected outcome, not guarantees. Users who treat a 70% prediction as a certainty will be disappointed 30% of the time.

Third, the model knows nothing about match-day specifics. It does not see the toss result, the playing XI, the pitch report, the weather, or the venue. All of these carry significant signal in T20 cricket. A production system would ingest these on the day of the match and adjust the pre-match probability accordingly.

9.2 Recommended Extensions

Three extensions would likely improve accuracy further. **Add player-level features.** The current system is team-level only. Tracking the availability of star players (Bumrah, Kohli, etc.) and their recent individual form would add significant signal.

Add venue features. Some grounds favour chasing (small boundaries, dew at night), others favour defending (large outfield, turning pitch). A team's record at the specific venue would add context.

Incorporate the toss. The toss decision (bat vs bowl first) is known 30 minutes before play and carries strong signal at venues where the dew factor matters. A pre-match adjustment based on the toss result would lift late-prediction accuracy substantially.

9.3 How to Use the System

To predict an upcoming match A vs B, run the following:

1. Update team states with the most recent match results
2. Compute the 13 features from the running statistics
3. Compute the 6 sub-probabilities (ELO, form, run-rate, win%, H2H, momentum)
4. Apply the optimised weights: 0.30, 0.10, 0.15, 0.15, 0.10, 0.20
5. If the result ≥ 0.5 , predict team A; else predict team B
6. After the match, update states and ELO with the result, then advance

The accompanying Python script (`ipl_predict_enhanced.py`) implements this end-to-end and can be re-run with new match data to refresh the team states and produce fresh predictions.

Deliverables accompanying this report: `ipl_predictions.json` (full per-match predictions for all 11 systems), `chart_model_comparison.png`, `chart_cumulative_acc.png`, `chart_elo_trajectory.png`, `chart_final_elo.png`, and `ipl_predict_enhanced.py` (the recoverable analysis script).